

Assignment 3: Inventory and Data Management System

Course	Web Dev III
Unit Covered	Unit – 3
CO Mapping	CO3
Marks	2.5
Mode	100% In-Class (Lab Practicals)
Time	30–40 minutes

Assignment Overview

The Inventory and Data Management System is a hands-on lab designed to strengthen your understanding of database-driven backend development using Node.js, Express.js, and MongoDB. In this lab, you will build a RESTful backend that allows a store or warehouse to manage its product catalogue — creating products, updating stock levels, searching and filtering the catalogue, and generating inventory reports through schema validation and aggregation queries.

This assignment focuses on persistent storage, schema design, data validation, and advanced querying techniques that are essential for any full-stack or backend developer working with real-world, data-heavy applications.

Learning Objectives

- Design a well-structured Mongoose schema with field-level validation
- Connect an Express.js application to MongoDB using Mongoose
- Implement complete CRUD operations for a Product/Inventory resource
- Apply query-string based filtering, sorting, and pagination
- Use MongoDB aggregation pipelines to generate inventory reports
- Handle stock increments/decrements safely (restock and sale operations)
- Implement centralized error handling and input validation middleware
- Manage environment variables securely using dotenv

Technology Stack

- Node.js – Server-side runtime environment
- Express.js – Backend routing and middleware
- MongoDB – NoSQL document database for persistent storage
- Mongoose – Schema definition, validation, and querying (ODM)
- dotenv – Environment variable management

Restrictions:

- No pre-built admin panel, CMS, or inventory-management boilerplate
- Only Node.js, Express, Mongoose, and dotenv are allowed

Functional Requirements

- Product Schema & Validation
- Create Product API
- Get All Products — Filtering, Sorting & Pagination

- Get Single Product by ID
- Update Product Details & Stock Adjustment
- Delete Product
- Low-Stock Alert Report
- Category-wise Inventory Summary (Aggregation)

Expected Output

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/products`. The response is a 200 OK status with a response time of 98 ms and a body size of 3.31 KB. The response body is displayed in JSON format, showing a paginated list of products.

```
2  {"total": 12,
3  "page": 1,
4  "totalPages": 2,
5  "count": 10,
6  "products": [
7    {
8      "_id": "6a54767af6123427d6e61d76",
9      "name": "Study Table",
10     "sku": "FURN-TB-812",
11     "category": "Furniture",
12     "price": 3999,
13     "quantity": 2,
14     "reorderLevel": 4,
15     "supplier": "IKEA India",
16     "createdAt": "2026-07-13T05:24:10.351Z",
17     "updatedAt": "2026-07-13T05:24:10.351Z",
18     "__v": 0,
19     "stockValue": 7998,
20     "id": "6a54767af6123427d6e61d76"
--  }
```

POST ▼ http://localhost:5000/api/products Send ▼

Docs Params Authorization Headers (9) **Body** • Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 { "name": "Gel Pen Pack of 15", "sku": "STNY-PN-713", "category": "Stationery", "price": 120, "quantity": 40, "reorderLevel": 20,
  "supplier": "Cello Pens" }
```

Body Cookies Headers (7) Test Results ↺ **201 Created** • 19 ms • 552 B • 🌐 ⋮

{} **JSON** ▼ ▶ Preview 🖼️ Visualize ▼ 🔄 ☰ 🔍 📄 🔗

```
1 {
2   "name": "Gel Pen Pack of 15",
3   "sku": "STNY-PN-713",
4   "category": "Stationery",
5   "price": 120,
6   "quantity": 40,
7   "reorderLevel": 20,
8   "supplier": "Cello Pens",
9   "_id": "6a547727f6123427d6e61d7c",
10  "createdAt": "2026-07-13T05:27:03.054Z",
11  "updatedAt": "2026-07-13T05:27:03.054Z",
12  "__v": 0,
13  "stockValue": 4800,
14  "id": "6a547727f6123427d6e61d7c"
15 }
```

PUT ▼ http://localhost:5000/api/products/6a547727f6123427d6e61d7c Send ▼

Docs Params Authorization Headers (9) **Body** • Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 { "name": "Gel Pen Pack of 15", "sku": "STNY-PN-713", "category": "Stationery", "price": 140, "quantity": 60 }
```

Body Cookies Headers (7) Test Results ↺ **200 OK** • 131 ms • 547 B • 🌐 ⋮

{} **JSON** ▼ ▶ Preview 🖼️ Visualize ▼ 🔄 ☰ 🔍 📄 🔗

```
1 {
2   "_id": "6a547727f6123427d6e61d7c",
3   "name": "Gel Pen Pack of 15",
4   "sku": "STNY-PN-713",
5   "category": "Stationery",
6   "price": 140,
7   "quantity": 60,
8   "reorderLevel": 20,
9   "supplier": "Cello Pens",
10  "createdAt": "2026-07-13T05:27:03.054Z",
11  "updatedAt": "2026-07-13T05:30:25.913Z",
12  "__v": 0,
13  "stockValue": 8400,
14  "id": "6a547727f6123427d6e61d7c"
15 }
```

DELETE ⌵ http://localhost:5000/api/products/6a547727f6123427d6e61d7c **Send** ⌵

Docs **Params** Authorization Headers (9) **Body** • Scripts Tests Settings Cookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

Body Cookies Headers (7) Test Results ↺ **200 OK** • 289 ms • 296 B • 🌐 ⋮

{} **JSON** ⌵ ▶ Preview 🖼 Visualize ⌵ 🔄 ☰ 🔍 📄 🔗

```
1 {
2   "message": "Product deleted",
3   "id": "6a547727f6123427d6e61d7c"
4 }
```

GET ⌵ http://localhost:5000/api/products/low-stock **Send** ⌵

Docs **Params** Authorization Headers (9) **Body** • Scripts Tests Settings Cookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

Body Cookies Headers (7) Test Results ↺ **200 OK** • 153 ms • 1.46 KB • 🌐 ⋮

{} **JSON** ⌵ ▶ Preview 🖼 Visualize ⌵ 🔄 ☰ 🔍 📄 🔗

```
1 {
2   "count": 4,
3   "lowStockItems": [
4     {
5       "_id": "6a54767af6123427d6e61d76",
6       "name": "Study Table",
7       "sku": "FURN-TB-812",
8       "category": "Furniture",
9       "price": 3999,
10      "quantity": 2,
11      "reorderLevel": 4,
12      "supplier": "IKEA India",
13      "createdAt": "2026-07-13T05:24:10.351Z",
14      "updatedAt": "2026-07-13T05:24:10.351Z",
15      "__v": 0,
16      "stockValue": 7998,
17      "id": "6a54767af6123427d6e61d76"
18    },
19    {
20      "name": "Study Table",
21      "sku": "FURN-TB-812",
22      "category": "Furniture",
23      "price": 3999,
24      "quantity": 2,
25      "reorderLevel": 4,
26      "supplier": "IKEA India",
27      "createdAt": "2026-07-13T05:24:10.351Z",
28      "updatedAt": "2026-07-13T05:24:10.351Z",
29      "__v": 0,
30      "stockValue": 7998,
31      "id": "6a54767af6123427d6e61d76"
32    }
33   ]
34 }
```

GET Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (7) Test Results

200 OK • 157 ms • 733 B •

{ } JSON Preview Visualize

```
1 {
2   "categories": 5,
3   "summary": [
4     {
5       "_id": "Electronics",
6       "totalItems": 4,
7       "totalQuantity": 96,
8       "totalStockValue": 343904,
9       "avgPrice": 5636.5
10    },
11    {
12      "_id": "Apparel",
13      "totalItems": 2,
14      "totalQuantity": 78,
15      "totalStockValue": 75522,
16      "avgPrice": 1399
17    }
18  ]
19 }
```

PATCH Send

Docs Params Authorization Headers (9) Body Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "change":10
3 }
```

Body Cookies Headers (7) Test Results

200 OK • 137 ms • 578 B •

{ } JSON Preview Visualize

```
1 {
2   "message": "Stock updated",
3   "product": {
4     "_id": "6a54767af6123427d6e61d76",
5     "name": "Study Table",
6     "sku": "FURN-TB-812",
7     "category": "Furniture",
8     "price": 3999,
9     "quantity": 12,
10    "reorderLevel": 4,
11    "supplier": "IKEA India",
12    "createdAt": "2026-07-13T05:24:10.351Z",
13    "updatedAt": "2026-07-13T05:35:24.419Z",
14    "__v": 0,
15    "stockValue": 47988,
16    "id": "6a54767af6123427d6e61d76"
17  }
18 }
```

Concepts Covered

Mongoose Schema Design, Field-Level Validation, RESTful CRUD, Query Parameters, Filtering, Sorting, Pagination, Aggregation Pipelines, Middleware, Centralized Error Handling, dotenv Configuration.

Rubric for Assignment – 3

Criteria	Marks
Database Schema Design & Validation (Mongoose)	0.5
CRUD Functionality (Create, Read, Update, Delete)	0.75
Querying — Filtering, Sorting, Pagination	0.5
Aggregation — Low-Stock Report & Category Summary	0.5
API Design & Clean Code	0.25
Total	2.5

Happy Coding!